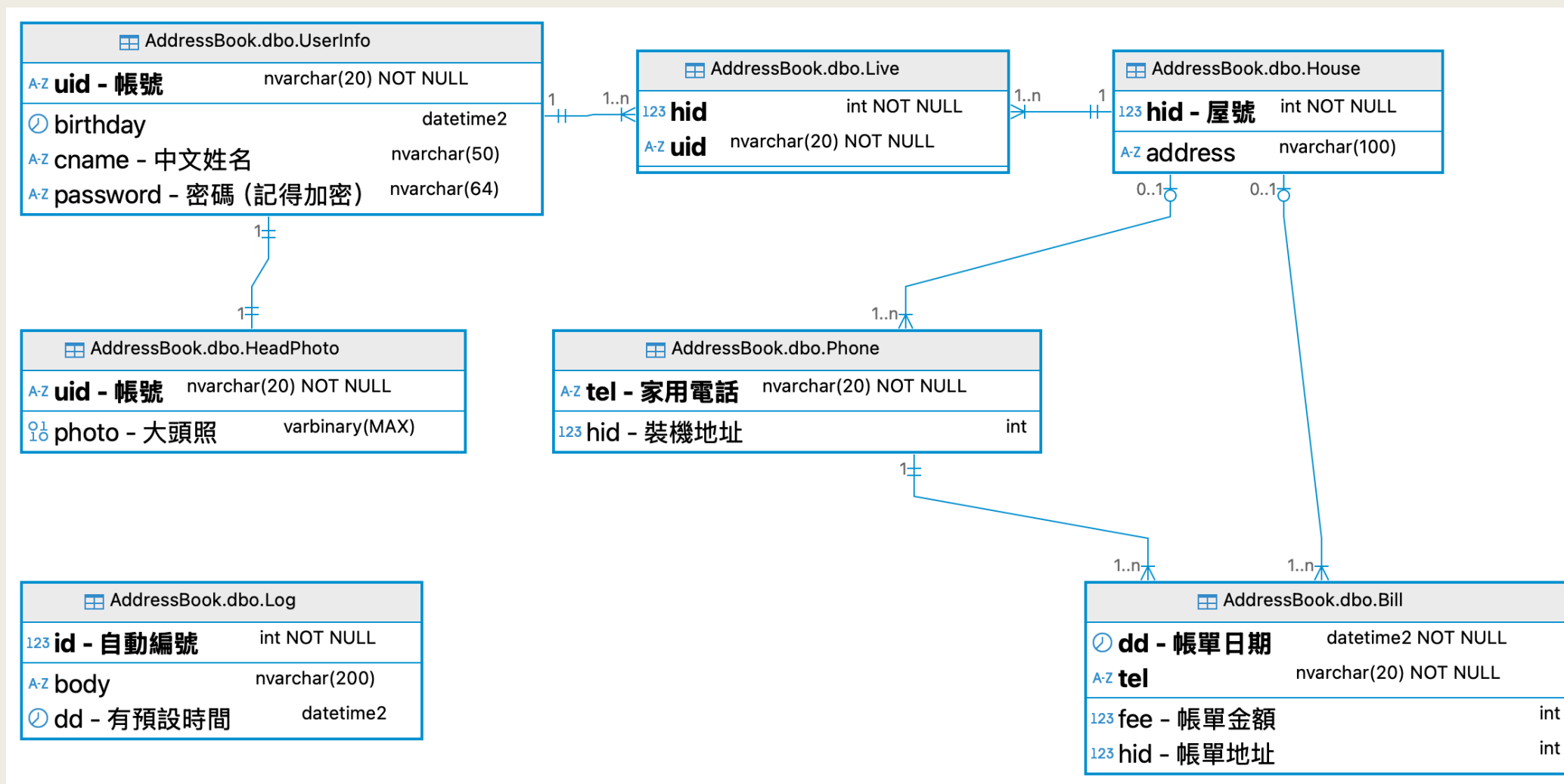


SQL COMMAND

朱克剛



ERD + Data Dictionary



查詢語法

- SQL 指令大小寫不分，資料查詢時是否分大小寫要看資料庫定序
 - 定序中有 CS 表示分大小寫
 - 定序中有 CI 表示不分大小寫

select 欄位
from 資料表
where 查詢條件

沒有條件時，WHERE 可不寫

最簡單的查詢

- 查詢王大明在UserInfo資料表中的所有資料

```
select *  
from UserInfo  
where cname = '王大明'
```

- 可以只列出需要的欄位，例如只要顯示帳號與生日

```
select uid, birthday  
from UserInfo  
where cname = '王大明'
```

邏輯合併 – and / or

- 查詢姓名為王大明或李大媽的資料

```
select *  
from UserInfo  
where cname = '王大明' or cname = '李大媽'
```

- 查詢帳號為 A01，密碼為 1234 的基本資料

```
select *  
from UserInfo  
where uid = 'A01' and password = '1234'
```

包含運算 – in

- 查詢姓名為王大明或李大媽的資料

```
select *  
from UserInfo  
where cname = '王大明' or cname = '李大媽'
```

- 也可以這樣寫

```
select *  
from UserInfo  
where cname in ('王大明', '李大媽')
```

不包含運算 – not in

- 查詢除了王大明或李大媽以外的資料

```
select *  
from UserInfo  
where cname not in ('王大明', '李大媽')
```

- 上面的結果資料有錯，錯在哪？

NULL 處理 1/3

- 當資料內容為 NULL 時，表示這個欄位跳過沒輸入。由於 NULL 不屬於任何型態，所以 NULL 的查詢必須使用「is」而不是「=」
- 例如：查詢姓名不完整的資料（包含 NULL 與空字串）

```
select *  
from UserInfo  
where cname is null or cname = ''
```

NULL 處理 2/3

- 如果不希望查詢結果出現 NULL，可以使用函數給一個預設值
- SQL Server

```
select uid, isnull(cname, "")  
from UserInfo
```

- MySQL

```
select uid, ifnull(cname, "")  
from UserInfo
```

NULL 處理 3/3

- 查詢除了王大明或李大媽以外的資料的正確指令

```
select *  
from UserInfo  
where cname not in ('王大明', '李大媽') or cname is null
```

模糊查詢 – like

- 模糊查詢也稱部分查詢，也就是條件並不精準，只知道其中一部分而已
- 例如：查詢姓王的資料

```
select *  
from UserInfo  
where cname like '王%'
```

- 只要 cname 中出現「妹」字的都要列出

```
select *  
from UserInfo  
where cname like '%妹%'
```

注意：這個查詢效率非常差

查詢欄位可以無中生有

- 查詢 Bill 資料表中的所有資料，增加一個欄位，計算帳單金額打 9 折後的資料

```
select *, fee * 0.9  
from Bill
```

- 取個別名讓後端好處理

```
select *, fee * 0.9 as discount  
from Bill
```

條件判斷 – case when

- 帳單金額如果超過 500 就打 8 折，超過 300 打 9 折

```
select *,  
  case  
    when fee > 500 then fee * 0.8  
    when fee > 300 then fee * 0.9  
    else fee  
  end as discount  
from Bill
```

欄位別名

比較運算

- 查詢帳單金額大於等於 500 的資料

```
select *  
from Bill  
where fee >= 500
```

- 查詢帳單金額不等於 500 的資料

```
select *  
from Bill  
where fee <> 500
```

範圍（內）運算 – between and

- 查詢帳單金額大於等於300小於等於700的資料

```
select *  
from Bill  
where fee between 300 and 700
```

- 相當於

```
select *  
from Bill  
where fee >= 300 and fee <= 700
```

範圍（外）運算 – not between and

- 查詢帳單金額低於 300 且超過 700 的資料

```
select *  
from Bill  
where fee not between 300 and 700
```

- 相當於

```
select *  
from Bill  
where fee < 300 or fee > 700
```

排序 – order by

- 根據帳單金額由小到大排序（稱為正向排序或順向排序）

```
select *  
from Bill  
order by fee
```

- 若要由大到小排序（稱為逆向排序或反向排序）

```
select *  
from Bill  
order by fee desc
```

只有數字才能比較大小

- 所有文字都必須轉成數字後才能比較大小，然後才能排序
- 英文與數字的排序方式是依照 ASCII 表來排

順向排序時：0 - 9 A - Z a - z

- <https://zh.wikipedia.org/zh-tw/ASCII>

排序方式與定序有關

- 對於非英文與數字的排序，結果會參考資料庫所設定的定序
- 將 cname 欄位按照筆畫數排序（SQL Server 預設筆畫數）

```
select cname  
from UserInfo  
where cname <> '' or cname is not null  
order by cname
```

- 如果是 MySQL 必須改定序才會得到同樣結果（MySQL 預設 Unicode）

```
select cname  
from UserInfo  
where cname <> '' or cname is not null  
order by convert(cname using big5)
```

合併資料表 – inner join

- 查詢王大明的帳號與住址（手上一定要有 ER）

```
select UserInfo.uid, cname, address
from UserInfo, Live, House
where
    UserInfo.uid = Live.uid and Live.hid = House.hid
    and cname = '王大明'
```

uid 前不加資料表名稱會得到
「模擬兩可的欄位」錯誤訊息

- 也可以這樣寫（inner 可省略）

```
select UserInfo.uid, cname, address
from UserInfo
    inner join Live
        on UserInfo.uid = Live.uid
    inner join House
        on Live.hid = House.hid
where cname = '王大明'
```

合併資料表 – outer join

- 查詢朱小妹的帳號與住址 (outer 可省略)
- 此種連結稱為左側外部連結

```
select UserInfo.uid, cname, address
from UserInfo
  left outer join Live
    on UserInfo.uid = Live.uid
  left outer join House
    on Live.hid = House.hid
where cname = '朱小妹'
```

- 若 from 後面的順序改為 House, Live, UserInfo，就需要使用右側外部連結

內部與外部連結的差異

- 內部連結：關連線兩端相同的資料合併，不相同的資料刪除

<u>uid</u>	cname
A01	王大明
A04	朱小妹

Inner Join

<u>uid</u>	<u>hid</u>
A01	1



uid	cname	uid	hid
A01	王大明	A01	1

- 外部連結：關連線兩端多的資料留下，缺少的補 NULL

<u>uid</u>	cname
A01	王大明
A04	朱小妹

Left Join

<u>uid</u>	<u>hid</u>
A01	1



uid	cname	uid	hid
A01	王大明	A01	1
A04	朱小妹	NULL	NULL

若只能選一種連結

- 連結方式只能選一種，在沒有把握的情況下，要選外部連結，資料才不會漏掉
- 查詢誰沒有屋子住

```
select *  
from UserInfo left join Live on UserInfo.uid = Live.uid  
where Live.hid is null
```

交叉連結 – cross join

- 只要關連忘記下就是交叉連結，查詢結果為每個資料表的數量乘積，資料量暴增。
- 大部分的交叉連結都是錯的，例如：

```
select *  
from UserInfo, Live
```

<u>uid</u>	cname
A01	王大明
A02	李大媽
A04	朱小妹

<u>uid</u>	<u>hid</u>
A01	1
A02	1



uid	cname	uid	hid
A01	王大明	A01	1
A01	王大明	A02	1
A02	李大媽	A01	1
A02	李大媽	A02	1
A04	朱小妹	A01	1
A04	朱小妹	A02	1

群組運算

- 查詢每支電話的帳單金額總和

```
select tel, sum(fee) as sum_fee  
from Bill  
group by tel
```

- 查詢每支電話的帳單金額總和並且列出裝機地址

```
select b.tel, sum(fee) as sum_fee, address  
from Bill as b, Phone as p, House as h  
where b.tel = p.tel and p.hid = h.hid  
group by b.tel, address
```

欄位別名



tel	fee
1111	300
1111	700
1111	900
2222	150
2222	400

資料表別名



tel	fee	address
1111	300	台北市
1111	700	台北市
1111	900	台北市
2222	150	新竹市
2222	400	新竹市

群組運算函數

- 常用的函數：總和 sum()、平均 avg()、最大值 max()、最小值 min()、筆數 count()
- 如果沒有 group by 指令，全部資料算一個群組
- 例如：Bill 資料表中共有幾筆資料？

```
select count(*) from Bill
```

- 帳單最高的金額是多少？

```
select max(fee) from Bill
```

子查詢 1/3

- 子查詢 (subquery) 又稱為巢狀查詢，意思是一個查詢指令中有多個 select 指令
- 第一階段：列出姓名中的姓氏

```
select left(cname, 1)
from UserInfo
where cname <> '' and cname is not null
```

- 第二階段：計算每個姓氏有幾筆資料

```
select lastname, count(*) as n
from (
    select left(cname, 1) as lastname
    from UserInfo
    where cname <> '' and cname is not null
) as tmp
group by lastname
```

子查詢 2/3

- 計算空屋率

- 第一階段：佈局

select 沒人住的房屋數量 / 全部房屋數量

- 第二階段：計算分母

select count(*) from House

- 第三階段：計算分子

select count(*)
from Live **right join** House **on** Live.hid = House.hid
where Live.uid is null

- 第四階段：組合

子查詢 3/3

- 第四階段：組合

```
select (  
    select count(*)  
    from Live right join House on Live.hid = House.hid  
    where Live.uid is null  
)/ cast( (select count(*) from House) as float)
```

- 注意：SQL Server 的除法運算使用「整數除整數結果為整數」的機制

```
select 5 / 3 -- 結果為 1
```

極端值查詢 1/2

- 意思是資料排序後查詢兩個端點的資料
- 查詢帳單金額最多的那筆資料
- SQL Server

```
select top 1 * from Bill  
order by fee desc
```

- MySQL

```
select * from Bill  
order by fee desc  
limit 1
```

極端值查詢 2/2

- 使用 top 1 或是 limit 1 的作法簡單、粗暴，但可惜資料會漏掉。當極端值有兩筆以上同燈同分時，查詢結果只會出現一筆
- 正確作法

```
select * from Bill
where fee = (
    select max(fee) from Bill
)
```

- SQL Server 有專屬指令

```
select top 1 with ties * from Bill
order by fee desc
```

群組後條件 – having

- 查詢電話帳單金額總和大於等於 1000 的資料

```
select tel, sum(fee) as sum_fee
from Bill
group by tel
having sum(fee) >= 1000
```

- 也可以使用子查詢

```
select * from (
    select tel, sum(fee) as sum_fee
    from Bill
    group by tel
) as tmp
where sum_fee >= 1000
```

輸出 JSON

- JSON 是常見的一種資料交換格式，如果採前後端分離架構或撰寫 Web API 一定會用到 <https://www.json.org>

- SQL Server

```
select *  
from UserInfo  
for json path, include_null_values
```

- MySQL

```
select concat('[', group_concat(  
    json_object(  
        'uid', uid,  
        'cname', cname  
    )  
, ', ']) from UserInfo
```

救命指令 – distinct

- 當資料重複時，可以使用 distinct 刪除多餘資料
- 例如：從帳單資料表中找出哪些電話有帳單資料

select distinct tel from Bill

- 救命處在於當資料重複卻又找不出指令錯在哪時，先用 distinct 應急一下

救命指令 – union [all]

- 垂直合併兩個查詢結果，限制是第二個查詢欄位結構必須對齊第一個查詢
- 臨時需要一筆帳單金額為 0 的資料作為測試，但又不想實際修改 Bill 資料表

```
select * from Bill
union all
select '9999', 0, 1, getdate()
```

- 若沒有 all，已經有的資料就不會合併
- 救命處在於發現資料有缺，但不知道指令錯在哪時，先補一筆缺少的資料再說

建立 View

- View 又稱視觀表或檢視表，其實就是將查詢指令給個名字並存在資料庫中
- 例如將帳單金額最高的查詢建立 view

```
create view vw_max_fee as  
select * from Bill  
where fee = (  
    select max(fee) from Bill  
)
```

- 使用方式

```
select * from vw_max_fee
```

View 的優點一大堆

- 少了語法檢查時間，執行速度比直接下 SQL 指令快一點
- 有效屏蔽資料庫結構，對資訊安全有幫助
- 可在 view 中透過 where 條件以及 select 欄位控制資料揭露程度
- 建立時若沒下特殊指令，view 為唯讀
- 後端工程師不用下一堆 SQL 指令 🤔，他會很開心 ❤️ 👍

插入資料

- 插入一筆資料到 UserInfo 資料表 (所有欄位資料都需準備)

```
insert into UserInfo values ('B01', 'David', NULL, NULL)
```

- 只準備需要的欄位資料

```
insert into UserInfo (uid, cname) values ('B02', 'Betty')
```

- 有自動編號的欄位必須排除

```
insert into House (address) values ('花蓮市月眉路1號')
```

修改資料

- 將所有密碼全部改為 1234 (沒有 where 條件別亂下!!!)

```
update UserInfo set password = '1234'
```

- 將 B01 的密碼改為 5678，生日改為 '2000/1/1' (欄位間用逗點隔開) 

```
update UserInfo set  
    password = '5678',  
    birthday = '2000/1/1'  
where uid = 'B01'
```

刪除資料

- 刪除所有 Live 資料表中的資料 (沒有 where 條件別亂下!!!)

delete from Live

- 刪除全部資料的另一種下法 (絕對別亂下!!!)

truncate table Live

- 刪除 B01 的資料 

delete from UserInfo **where** uid = 'B01'